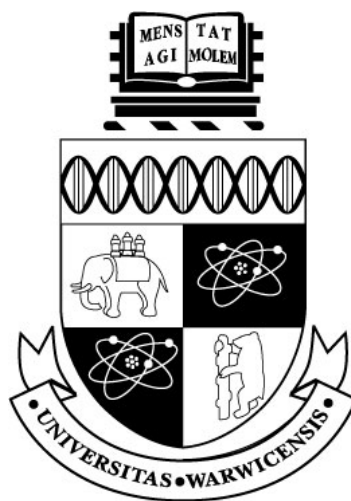




# GODOT 4.6 大型可视化行为树的显示优化与实验评估

by

[作者姓名]

A thesis submitted in partial fulfilment  
of the requirements for the degree of

游戏工程理学硕士

UNIVERSITY OF WARWICK

华威大学 WMG

2026 年 9 月

# 目录

|                           |          |
|---------------------------|----------|
| 目录                        | ii       |
| 插图                        | iv       |
| 表格                        | v        |
| 致谢                        | vi       |
| 声明                        | vii      |
| 摘要                        | viii     |
| <b>1 引言</b>               | <b>1</b> |
| 1.1 研究背景 . . . . .        | 1        |
| 1.2 问题定义与研究空白 . . . . .   | 1        |
| 1.3 研究目标与问题 . . . . .     | 2        |
| 1.4 研究目标与成功标准 . . . . .   | 2        |
| <b>2 文献综述</b>             | <b>3</b> |
| 2.1 作为可执行层级的行为树 . . . . . | 3        |
| 2.2 节点-连线树与扩展性 . . . . .  | 3        |
| 2.3 焦点、上下文与概览 . . . . .   | 4        |
| 2.4 复杂度管理与心理地图 . . . . .  | 4        |
| 2.5 运行时解释与失败定位 . . . . .  | 5        |
| 2.6 研究空白与设计启示 . . . . .   | 5        |
| <b>3 研究方法</b>             | <b>6</b> |
| 3.1 研究策略 . . . . .        | 6        |
| 3.2 实验因素与数据集 . . . . .    | 6        |
| 3.3 受控多规模显示实验 . . . . .   | 7        |
| 3.4 物理屏幕尺寸实验 . . . . .    | 8        |
| 3.5 插件功能与可玩游戏验证 . . . . . | 8        |
| 3.6 有效性与可复现性 . . . . .    | 8        |

|          |                            |           |
|----------|----------------------------|-----------|
| <b>4</b> | <b>系统设计与实现</b>             | <b>9</b>  |
| 4.1      | 架构与项目边界 . . . . .          | 9         |
| 4.2      | 资源模型与校验 . . . . .          | 9         |
| 4.3      | 运行时语义 . . . . .            | 10        |
| 4.4      | 编辑器编写工作流 . . . . .         | 10        |
| 4.5      | Live Debug 与黑板检查 . . . . . | 11        |
| 4.6      | 大型树显示机制 . . . . .          | 11        |
| 4.6.1    | 信息密度控制 . . . . .           | 12        |
| 4.6.2    | 焦点与结构裁剪 . . . . .          | 12        |
| 4.6.3    | 导航与概览 . . . . .            | 13        |
| 4.6.4    | 连线与运行表示 . . . . .          | 13        |
| 4.7      | 物理屏幕实验基础设施 . . . . .       | 14        |
| 4.8      | 演示游戏 . . . . .             | 14        |
| <b>5</b> | <b>结果与分析</b>               | <b>15</b> |
| 5.1      | 插件功能与可玩游戏验证 . . . . .      | 15        |
| 5.2      | 受控多规模显示结果 . . . . .        | 15        |
| 5.3      | 物理屏幕尺寸结果 . . . . .         | 16        |
| 5.4      | 对应研究问题的结果 . . . . .        | 17        |
| <b>6</b> | <b>讨论</b>                  | <b>18</b> |
| 6.1      | 实验平台的功能正确性 . . . . .       | 18        |
| 6.2      | 显示优化效果评估 . . . . .         | 18        |
| 6.2.1    | 不同节点规模下的优化效果 . . . . .     | 18        |
| 6.2.2    | 不同物理屏幕尺寸下的优化效果 . . . . .   | 19        |
| <b>7</b> | <b>结论</b>                  | <b>20</b> |
| 7.1      | 总结 . . . . .               | 20        |
| 7.2      | 贡献 . . . . .               | 20        |
| 7.3      | 局限 . . . . .               | 21        |
| 7.4      | 未来工作 . . . . .             | 21        |
| 7.5      | 最终结论 . . . . .             | 21        |
|          | <b>参考文献</b>                | <b>22</b> |
| <b>A</b> | <b>六种实验条件参考</b>            | <b>24</b> |
| <b>B</b> | <b>复现与证据索引</b>             | <b>25</b> |
| <b>C</b> | <b>初稿完成清单</b>              | <b>26</b> |

# 插图

|     |                                                       |    |
|-----|-------------------------------------------------------|----|
| 4.1 | Godot 编辑器、行为树资源、Runner 与 Actor 之间的插件架构和数据流。 . . . . . | 9  |
| 4.2 | 仅位于编辑器中的活动路径和失败原因显示。 . . . . .                        | 11 |
| 4.3 | 同一行为树的完整细节基线和紧凑显示。 . . . . .                          | 12 |
| 4.4 | 局部放大和结构裁剪的两种实现。 . . . . .                             | 12 |
| 4.5 | 复杂树中的小地图、路径摘要、导航控件和自动布局。 . . . . .                    | 13 |
| 4.6 | 连线组织和运行状态表示。 . . . . .                                | 14 |
| 5.1 | 五种受控规模下的密度、显著性和上下文降低。 . . . . .                       | 16 |

# 表格

- 3.1 六种显示实验条件及实际操作。 . . . . . 6
- 4.1 已实现的运行时节点语义。 . . . . . 10
- 5.1 最终核心自动回归。 . . . . . 15
- 5.2 五种资源规模的受控显示测量，降低比例相对 Baseline。 . . . . . 16
- 5.3 三种物理屏幕尺寸下的复杂树覆盖率。分辨率仅为审计元数据，不是实验处理。 . . . . . 17
- A.1 论文使用的六种显示实验条件。 . . . . . 24

# 致谢

此处预留给作者感谢导师，以及帮助测试软件和审阅论文的人。提交前应由作者自行完成最终文字。

# 声明

本论文作为本人申请游戏工程理学硕士学位的材料提交给华威大学。论文由本人完成，且未曾用于申请其他学位。除明确引用的资料外，本文所述软件、测试数据和分析均来自本项目。本文没有报告任何参与者实验结果。正式提交前，作者必须根据学校规定审阅并个性化本声明，包括如实说明生成式人工智能的使用方式。

# 摘要

可视化行为树常用于编写游戏智能体的决策，但随着节点数量以及运行时信息增加，传统节点-连线编辑器会变得难以检查。本文设计并评估了一个 Godot 4.6 编辑器插件，将完整、可执行的行为树系统与可复位的大树显示方法结合。运行时支持有序、响应式、随机、并行、重复和定时控制流，支持 Actor 方法调用、类型化黑板、Decorator 以及仅存在于编辑器中的运行检查。显示实验只比较六种条件：完整显示的 Baseline、缩小卡片的 Compact Cards、结合紧凑卡片与低细节语义缩放的 Optimized Overview、突出固定目标的 Optimized Search、只显示指定分支及必要上下文的 Subtree Focus，以及折叠部分子树的 Context Collapse。

评估先通过自动测试和一棵可玩的 241 节点行为树确认实验平台正确，再在 31 至 364 节点的确定性树和三种物理尺寸屏幕上，对六种条件进行重复的配对测量，考察卡片占用、可见上下文、目标定位及显示状态复位。结果显示，Compact Cards 与 Optimized Overview 在保留全部节点的同时减少了卡片占用；Optimized Search 能够稳定定位并突出目标；Subtree Focus 与 Context Collapse 能够减少当前显示的无关上下文。所有方法在测试中均保持节点位置、树结构和执行顺序不变。上述结果支持这些方法改善大型行为树的可测显示属性，但尚不能替代参与者实验对可读性和使用效率的验证。



# 1 引言

## 1.1 研究背景

现代游戏智能体需要执行复杂的行为逻辑。对于小型原型，把全部决策写在单一过程式脚本中是可行的；但随着状态和中断条件增加，控制流会越来越难以检查和修改。行为树（Behaviour Tree, BT）把决策逻辑表示为控制节点和叶节点组成的层级结构。复合节点决定如何选择子节点，条件和动作则把抽象决策结构连接到游戏状态与 Actor 代码。高层策略可以独立于移动、动画或战斗方法进行编辑，这是行为树在游戏开发中的主要价值之一 (Colledanchise and Ögren, 2018)。

传统节点-连线树能够直接表达父子拓扑，但会随宽度和深度增长而迅速占用空间。行为树卡片还可能显示参数、描述、Decorator、黑板键和执行状态，比只显示标签的普通层级图更大。当数十或数百张卡片放入有限画布时，开发者必须频繁缩放、平移和搜索，当前执行路径也可能淹没在非活动分支中。这不只是美观问题，因为节点图正是设计者定位任务、理解顺序、修改条件和诊断失败的主要工具。

因此，本项目把一个功能完整的 Godot 行为树插件作为大型树显示研究的平台。在保持上下端口和“同级从左到右执行”的语义基础上，加入可独立开关的焦点 + 上下文、复杂度管理、导航和运行时解释技术。每项技术都能单独关闭，一方面防止视觉功能故障拖垮编辑器，另一方面保留可复现的基线比较条件。

## 1.2 问题定义与研究空白

树和图可视化领域已经提出许多方法。鱼眼和焦点 + 上下文视图会为焦点附近的元素分配更多空间 (Furnas, 1986; Lamping et al., 1995)；概览 + 细节保留全局参考视图 (Cockburn et al., 2008)；多列布局可以改善大扇出树的浏览 (Song et al., 2010)；增量展开和折叠可以降低图复杂度，同时尽量保持用户的心理地图 (Dogrusoz et al., 2018; Misue et al., 1995)；大型分层图研究还表明，路径任务的效率取决于具体表示和任务 (Bae and Watson, 2011)。

然而，这些研究没有直接回答游戏行为树编辑器应如何组合上述机制。行为树与文件目录等普通层级不同：同级顺序具有执行语义；叶节点会调用代

码；Decorator 可能阻止分支运行；图的状态会在每个运行 Tick 中变化。一个可用的方案必须保持执行顺序，支持编辑与撤销，公开黑板状态，并在不向游戏画面加入调试覆盖层的前提下解释失败。现有研究可以指导单项设计，但仍需要一个集成、可执行且可复现评估的 Godot 实现。

### 1.3 研究目标与问题

本文目标是在一个经过功能验证、能够驱动真实 NPC 的 Godot 4.6 可视化行为树平台上，设计并评估可组合的大型树显示优化。研究问题为：在不同的行为树节点规模和物理屏幕尺寸下，本项目提出的可组合显示优化，相较于传统基线显示，能否改善大型行为树的可测显示与编辑交互表现，同时保持行为树结构和执行语义不变？

### 1.4 研究目标与成功标准

项目工作分为平台建设与研究评估。平台建设包括：实现可序列化的树、节点、Decorator 和类型化黑板资源；实现 Root、Sequence、Selector、Reactive Selector、Random Selector、Parallel、Repeat、Wait、Action 和 Condition 的 SUCCESS、FAILURE 与 RUNNING 语义；允许可复用组件为 NPC 指定树和 Actor，并由叶节点调用 Actor 方法；提供创建、连接、断开、拖拽、类型化编辑、校验、保存、加载、撤销和重做；只在 Godot 编辑器中显示路径、节点状态、黑板和失败原因。

研究评估为大型树显示方法提供独立持久化开关和安全复位路径，并以原始数据比较不同节点规模、物理屏幕尺寸和优化前后条件下的几何显示、目标定位与上下文保留。在进行显示优化实验前，首先需要确认行为树插件的基础功能正确：各类节点能够按照预期执行，编辑后的行为树能够正确保存和重新打开，撤销与重做也不会破坏树的内容；可玩场景中的复杂敌人应当确实由 241 节点行为树控制。显示优化需要在真实渲染环境中与原始显示方式进行比较。关闭优化后，编辑器应当恢复原来的显示状态，并且不能改变行为树的结构、节点位置和执行顺序。测试过程中只要出现程序报错、崩溃、内存泄漏或异常退出，就认为测试没有通过，即使其他检查项目显示成功也是如此。

## 2.1 作为可执行层级的行为树

行为树会重复评估一棵有根层级，并传播 `SUCCESS`、`FAILURE` 或 `RUNNING` 三种状态。`Sequence` 通常在第一个失败或运行中的子节点停止；`Selector` 在第一个成功或运行中的子节点停止；叶节点读取状态或执行动作。这个小型状态接口使复杂逻辑能够组合，并能明确表达中断策略 (Colledanchise and Ögren, 2018)。

三状态接口看似简单，却包含关键实现选择。运行中的 `Sequence` 可以记住当前索引，而 `Reactive Selector` 需要重新检查高优先级条件并抢占较低优先级分支；`Random Selector` 在子节点运行期间应保持同一次选择；`Parallel` 需要成功和失败阈值；`Repeat` 与 `Wait` 需要在重启或中断时清除记忆。因此，编辑器显示和运行语义不能完全分离：子节点顺序、Decorator 归属和节点身份都会影响执行，而且必须正确序列化。

黑板在感知、条件和动作之间提供共享键值状态。Unreal Engine 的工作流说明了把黑板、Decorator 和运行观察结合的价值 (Epic Games, 2026)。但本项目没有必要重现商业系统的全部能力。合理基线是：系统能够表达有意义的决策、调用项目 Actor 方法、验证黑板键，并解释分支为何运行或被拒绝。

## 2.2 节点-连线树与扩展性

节点-连线图适合行为树，因为能直接表示祖先关系和同级分组。经典整洁树算法追求层级一致、子树不重叠且相对紧凑的布局 (Reingold and Tilford, 1981)。`SpaceTree` 又把节点-连线结构与动态布局、缩放和选择性展开结合，其评估说明表示和交互会共同影响拓扑理解与重访任务 (Plaisant et al., 2002)。

问题在于规模。卡片占据面积，随着宽度和深度增加，显式连线也会变密。Song 等比较传统、列表和多列视图，发现多列界面在其大扇出层级的浏览和理解任务中更快，总体偏好也更高 (Song et al., 2010)。这一结果适用于包含大量备选行为的 `Selector` 或 `Sequence`，但不能直接照搬：论文中的子节点按字母排序，而行为树的横向顺序可能决定优先级。因此多列行为树必须明确显示顺序，且不能在布局时静默修改资源。

Bae 与 Watson 为大型分层图提出 `Quilts` 表示 (Bae and Watson, 2011)。其价值不一定是要求行为树完全改用矩阵，而是证明路径任务可能更适合另一种补

充表示。本项目因此保留可编辑节点图，同时加入紧凑文字路径和强活动路径编码，以针对运行时路径定位任务。

### 2.3 焦点、上下文与概览

Furnas 用兴趣度函数形式化了广义鱼眼视图：综合元素先验重要性和与焦点的距离，决定显示显著程度 (Furnas, 1986)。Lamping 等使用双曲几何在有限区域显示大型层级，对焦点进行放大并压缩远处上下文 (Lamping et al., 1995)。行为树编辑器可以借此读取鼠标所在节点，同时保留周边结构。

但是焦点 + 上下文并不天然优于其他方法。畸变会移动目标并增加点击难度，缩放变化也可能破坏心理地图。Cockburn 等区分概览 + 细节、缩放和焦点 + 上下文，并指出效果取决于任务、实现和切换成本 (Cockburn et al., 2008)。因此本项目把鱼眼最大倍率限制为 1.20，改变卡片内部表示而不是施加不稳定的整体图变换，以节点中心补偿位置，并在关闭时恢复全部几何。同时提供独立固定小地图，使两类技术可以比较或组合。

语义缩放与几何缩放不同。几何缩放改变物理大小，语义缩放改变显示属性。低细节卡片保留类型颜色、图标和短标题；高细节卡片展示方法、参数、描述和 Decorator。它以信息完整性换取密度，却不改变执行图几何。此前插件曾把滚轮与临时节点缩放耦合，造成滚动时节点先缩小再放大；分离信息可见性和图几何后，该问题被消除。

### 2.4 复杂度管理与心理地图

当不需要同时查看全部节点时，展开-折叠和隐藏-显示可以降低复杂度。Dogrusoz 等提出大型网络的增量复杂度管理方法，包括展开、折叠和布局更新 (Dogrusoz et al., 2018)。该研究与心理地图原则一致：如果一次编辑引起大量无关节点移动，用户就必须重新学习位置 (Misue et al., 1995)。行为树折叠应保留折叠根，报告隐藏内容，并尽量不移动无关分支。

子树折叠和子树聚焦解决不同问题。折叠用摘要替代后代，同时保留树的其他部分；聚焦只显示目标分支和必要上下文。二者都可能隐藏运行节点，因此插件提供隐藏节点数量、两层摘要、路径指示以及显式展开/聚焦控制，而不是无提示地删除视图信息。

搜索高亮与非活动分支淡化属于较柔和的复杂度管理：几何保持不变，只降低非相关内容显著性。这符合 Shneiderman 的“先概览、再缩放和过滤、按需细节”原则 (Shneiderman, 1996)。形状/图标和色盲友好配色则提供冗余通道，避免只用颜色表达节点类型。

## 2.5 运行时解释与失败定位

实时调试为静态层级增加时间维度。只高亮叶节点不能解释其到达路径；高亮所有历史节点又会混淆当前状态。本项目记录有序的 Root 到叶节点链条、逐节点状态和当前失败解释。非活动分支可以淡化，独立路径摘要在图缩小时仍可阅读。

Decorator 让失败解释更加重要。Action 本身可能正确，却因为黑板比较、冷却或时间限制而从未执行。在所属节点上显示 Decorator 摘要并附加失败原因，可以把运行证据连接回编写表示；实时黑板进一步显示 Key、运行时类型、值和 Schema 状态。这体现了分层图研究的任务导向原则：调试时用户往往在问“为什么这个分支没有运行”，而不是“完整拓扑是什么”。

## 2.6 研究空白与设计启示

文献支持多个单项机制，却没有规定如何把它们集成到一个可执行 Godot 行为树编辑器中。本文据此采用四项原则：保留节点-连线拓扑和明确同级顺序作为基线；分离几何放大、语义细节和结构过滤；在折叠与布局中维护空间上下文，并在隐藏细节时提供概览或路径表示；最后，把几何效果和软件正确性与人的任务表现分开评估。最后一项尤其重要：面积或可见节点减少只能证明表示更紧凑，不能证明更容易理解。

# 3 研究方法

## 3.1 研究策略

本研究先实现一个能够正常编写和运行行为树的 Godot 4.6 插件，再用该插件测试大型行为树的显示优化。研究重点不是行为树是否适合制作游戏，而是这些显示优化能否在不同节点数量和屏幕尺寸下改善可测的界面表现。

研究分为五个阶段。第一步根据导师意见和论文要求确定研究问题；第二步实现行为树运行时和游戏演示；第三步完成编辑、保存、撤销和实时调试功能；第四步加入能够独立开启和关闭的显示优化；第五步使用固定行为树和自动脚本比较优化前后的结果。每次加入新功能后都重新运行测试，确认旧功能没有被破坏。

## 3.2 实验因素与数据集

实验使用五棵自动生成的行为树，资源节点数量分别为 31、61、121、241 和 364。它们覆盖小型行为树、超过导师所提约 50 节点问题的行为树，以及更大的压力测试场景。五棵树采用相同的生成规则、节点类型比例和 Decorator 比例，主要差别只有规模。Decorator 附着在其他节点上，不单独显示为画布卡片，因此五棵树分别显示为 26、51、101、202 和 305 张卡片。

项目还有一棵实际控制游戏敌人的 241 节点行为树。它包含治疗、近战、远程攻击、跳跃、攀爬、追击、搜索、撤退、巡逻和待机等行为。这棵树只用于确认插件能够正确控制复杂敌人，不作为单独的显示优化实验。

本论文只比较表 3.1 所列六种显示条件。每次试验开始前都先恢复同一初始状态：展开全部节点、清空搜索、取消聚焦、将缩放设为 100%，并关闭本实验涉及的显示优化。随后只应用当前条件规定的操作。

表 3.1: 六种显示实验条件及实际操作。

| 条件       | 实验中的具体操作                  | 界面变化与主要测量目的           |
|----------|---------------------------|-----------------------|
| Baseline | 关闭本实验涉及的显示优化，显示完整卡片和全部分支。 | 提供原始显示结果，作为其他条件的比较基准。 |

| 条件                 | 实验中的具体操作                          | 界面变化与主要测量目的                   |
|--------------------|-----------------------------------|-------------------------------|
| Compact Cards      | 只开启紧凑卡片。所有节点仍然显示，但卡片变小，只保留类型和短标题。 | 测量在不隐藏节点的情况下，卡片总面积减少多少。       |
| Optimized Overview | 同时开启紧凑卡片和低细节语义缩放，并将画布缩放到50%。      | 保留全部节点，但减少每张卡片的信息，用于查看整棵树的结构。 |
| Optimized Search   | 在 Overview 状态下搜索一个预先指定且名称唯一的节点。   | 突出目标、淡化其他节点，并检查目标是否进入当前视口。    |
| Subtree Focus      | 选择一个预先指定的分支作为聚焦目标。                | 只显示目标分支和必要上下文，减少当前画面中的无关节点。   |
| Context Collapse   | 保留通往固定目标的路径，并折叠路径以外的部分子树。         | 保留折叠根，用摘要代替其后代，减少同时显示的节点。     |

六种条件只改变卡片的大小、信息量、显著程度或当前显示范围。即使使用 Collapse，插件也只保存折叠状态，不会改变节点的父子关系或执行顺序。

实验记录以下数据：显示了多少卡片、卡片总面积、卡片保留了多少信息、Search 淡化了多少非目标节点、搜索目标是否进入视口，以及 Focus 和 Collapse 减少了多少当前上下文。其他显示功能不参加本论文的条件比较。这些数据只能说明界面的显示变化，不能直接证明用户更容易理解行为树。

3.3 受控多规模显示实验

受控实验使用 NVIDIA GeForce RTX 5070 Laptop GPU、Godot 4.6 OpenGL Compatibility 渲染器和 1600×900 的固定测试画布。实验包含五种节点规模和六种显示条件。每一种“规模-条件”组合正式测试 30 次，因此一共得到 (5×6×30=900) 条数据。

每组正式测试前先运行三次，但不记录结果。这一步称为预热，用于减少首次加载和渲染带来的波动。正式测试会轮换树规模和显示条件的顺序，避免某个条件总是先运行或后运行。每次测试都从相同状态开始：展开全部节点、清空搜索、取消聚焦、使用 100% 缩放，然后只应用当前需要测试的条件。Search 和 Focus 使用预先指定的固定目标，避免人工选择影响结果。

实验开始前规定了最低合格标准。Baseline 必须显示全部卡片，显示优化不能改变保存位置和执行顺序。Compact 的卡片总面积至少应减少 40%，Overview

至少减少 70%；61 节点以上的 Search 至少应淡化 95% 的非目标卡片；121 节点以上的 Focus 和 Collapse 至少应减少 70% 的显示卡片。

### 3.4 物理屏幕尺寸实验

第二组实验只比较屏幕的物理尺寸，不比较分辨率。Windows 的扩展显示识别数据（Extended Display Identification Data, EDID）显示，三台屏幕的物理尺寸分别为 34×22 cm、60×33 cm 和 70×39 cm。实验把每厘米统一换算为 35 个逻辑单位，因此三块屏幕对应 1190×770、2100×1155 和 2450×1365 的测试画布。分辨率、刷新率和系统缩放只作为设备记录，不参加比较。

五棵行为树和六种显示条件都会在台屏幕上测试。每种“屏幕-规模-条件”组合先预热两次，再正式记录三次，因此一共得到  $(3 \times 5 \times 6 \times 3 = 270)$  条数据。

该实验主要记录每块屏幕能够容纳的卡片比例、行为树面积与屏幕面积的比例、卡片面积、搜索目标是否进入视口，以及 Focus 和 Collapse 减少的上下文。实验仍然检查节点位置和执行顺序没有改变。结果只能说明不同屏幕尺寸带来的几何显示差异，不能直接证明大屏幕更容易使用。

### 3.5 插件功能与可玩游戏验证

在进行显示实验前，先确认插件本身能够正常工作。自动测试分为资源与运行时、编辑器界面、基础游戏和复杂竞技场四类，覆盖节点执行、Decorator、Actor 方法调用、创建与连接、保存与重新加载、撤销与重做，以及玩家和敌人的基本交互。

可玩场景还用于确认 241 节点行为树能够实际控制敌人的索敌、防御、近战、远程攻击、跳跃、攀爬、搜索、撤退和治疗。该部分只说明插件功能正确，并且能够用于非简单游戏场景，不作为显示优化效果的独立实验。

如果测试出现非零退出码、失败信息、脚本错误、内存泄漏、非法访问或程序崩溃，则整组测试判定为失败。只有测试中特意触发且结果符合预期的警告可以接受。

### 3.6 有效性与可复现性

实验使用固定行为树、固定目标和相同起始状态，显示功能也都可以独立关闭。这样可以减少测试顺序和人工选择对结果的影响。900 条多规模数据和 270 条屏幕数据都保留了原始 CSV、文件校验值和测试脚本，便于重复检查。

本研究仍有明确限制。卡片面积、覆盖率和字段数量测量的是界面显示，不是人的理解能力。测试也只在台 Windows 电脑、Godot 4.6、三台现有屏幕和一个二维游戏中完成。因此，结果不能直接推广到所有设备、所有游戏或所有开发者。



# 系统设计4与实现

## 4.1 架构与项目边界

系统分为编辑器、资源、运行时和测试四个部分，如图4.1所示。游戏通过资源文件和运行组件执行行为树，不依赖编辑器界面。Live Debug 也只出现在 Godot 编辑器中，不会加入最终游戏画面。

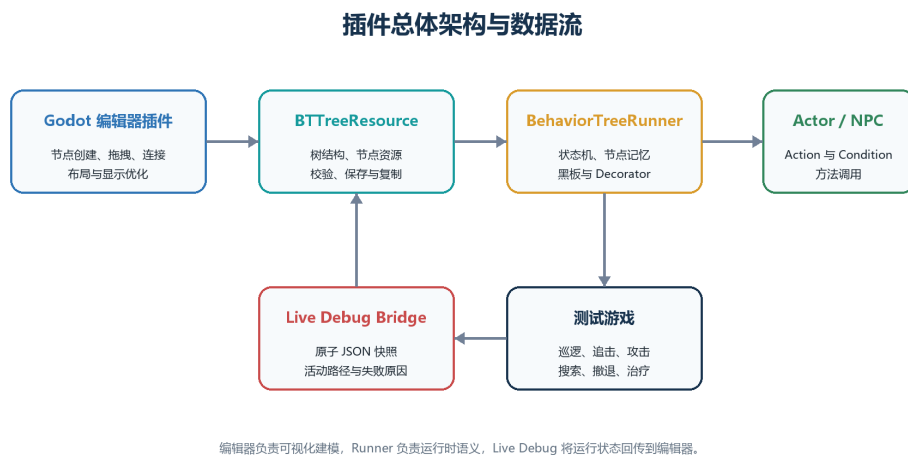


图 4.1: Godot 编辑器、行为树资源、Runner 与 Actor 之间的插件架构和数据流。

BTreeResource 保存整棵树，BTreeNodeResource 保存单个节点的类型、参数、位置和父级关系。BehaviorTreeRunner 负责执行行为树，BehaviorTreeComponent 则把行为树连接到场景中的 Actor。编辑器主界面负责画布、属性编辑、黑板、历史记录和 Live Debug。这样的分工使资源、执行逻辑和显示界面可以分别测试。

## 4.2 资源模型与校验

普通节点使用 parent\_id 记录父节点，Decorator 使用单独的 decorator\_parent\_id。因此 Decorator 不会被错误地当作普通子节点执行。同级子节点按画布上的横向位置排序，所以从左到右的显示顺序就是执行顺序。保存资源时，节点位置 and 这个顺序都会被保留。

保存前会检查 Root 是否存在、节点 ID 是否重复、父级是否有效、Decorator 是否正确附着，以及节点参数是否完整。例如，Root 不能有父级，Action 和 Condition 不能拥有普通子节点。编辑器保存和自动测试使用同一套检查规则。

黑板 Schema 可以声明 Bool、Int、Float、String 和 Vector2 类型，并为每个键设置默认值。严格模式会报告未声明或无效的键。Inspector 提供键选择器，同时保留自由输入方式。Schema 修改也进入撤销和重做历史。

4.3 运行时语义

行为树每次更新都会返回 SUCCESS、FAILURE 或 RUNNING。需要跨帧继续执行的节点会按 ID 保存进度；重启、切换行为树或中断分支时会清除这些记录。表4.1概括各类节点的行为。

表 4.1: 已实现的运行时节点语义。

| 节点                | 行为                           |
|-------------------|------------------------------|
| Root              | 执行唯一普通子节点。                   |
| Sequence          | 按序执行，遇失败或运行停止，并记忆运行索引。       |
| Selector          | 按序执行，遇成功或运行停止，并记忆运行索引。       |
| Reactive Selector | 每 Tick 从高优先级重新检查，可中断低优先级分支。  |
| Random Selector   | 使用可设种子的随机选择，运行期间保持同一子节点。     |
| Parallel          | Tick 多个子节点并应用成功/失败阈值。        |
| Repeat            | 固定次数或无限重复，跨 Tick 保存计数。       |
| Wait              | 累积 delta，到达时间后成功。            |
| Condition         | 调用 Actor 判断或比较黑板值。           |
| Action            | 调用 Actor 方法，把 Bool 或状态转成树状态。 |

Action 和 Actor 模式的 Condition 使用统一接口调用 Actor 方法。如果方法不存在，节点返回失败并显示原因，而不是使游戏崩溃。黑板条件支持键是否存在、真假、相等、不等和数值大小比较。

Decorator 会在所属节点执行前后改变条件或结果。已实现的功能包括黑板比较、冷却时间、时间限制、结果反转、强制结果和重复。重置子树时，Decorator 和子节点保存的运行进度会一起清除。

4.4 编辑器编写 workflow

插件显示在 Godot 编辑器底部。用户在画布上点击右键创建节点，不需要在菜单栏中面对一排创建按钮。节点可以移动、连接、断开、复制和删除。行为树选择器只显示资源名称，用户不需要手动查看或输入完整文件路径。创建、删除、移动、连线、参数修改、Schema 修改和折叠状态都支持撤销与重做。

Inspector 使用与字段类型对应的控件。例如, Action 直接填写方法名, Condition 选择 Actor 或黑板模式并设置键、比较方式和值。Random、Parallel、Repeat、Wait 和 Decorator 也有自己的设置项。高级 JSON 只作为兼容入口,不是正常操作的必需步骤。

本插件使用节点上方和下方的连接端口,而 Godot 默认连线主要面向左右端口。因此插件自行绘制正式连线,并在用户拖动连线时使用 Godot 的预览。自定义连线仍可通过右键断开,并支持撤销和重做。关闭自定义显示后可以恢复 Godot 原生连线。

## 4.5 LIVE DEBUG 与黑板检查

运行时会把当前 Actor、行为树、活动节点、执行路径、失败原因和黑板值写入调试文件。文件写完后才会替换旧版本,避免编辑器读到不完整内容。如果一次读取失败,编辑器保留上一份完整状态,并在下一次更新时重试。

编辑器只显示当前行为树的调试信息。活动路径使用明显边框,非活动分支可以淡化,失败节点会显示原因,黑板面板显示当前键和值。较长的路径放在可滚动区域中,避免遮挡主画布。游戏画面中不会出现这些调试内容。

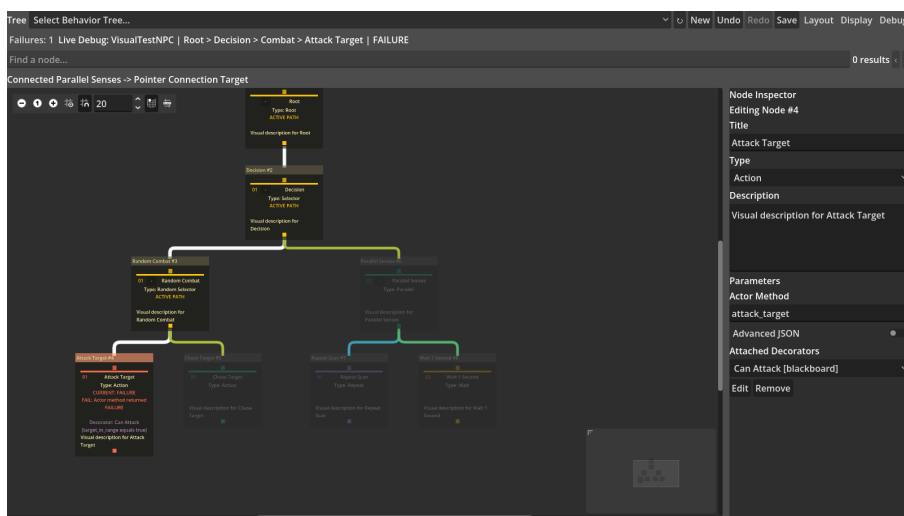


图 4.2: 仅位于编辑器中的活动路径和失败原因显示。

## 4.6 大型树显示机制

大型行为树的问题不只来自节点数量,还来自卡片信息、连线密度、导航距离和运行状态同时增加。为此,插件从信息密度、焦点与结构裁剪、导航与概览、连线与运行表示四个方面提供显示功能。

### 4.6.1 信息密度控制

Compact 缩小卡片，但不隐藏节点。卡片保留节点类型和短标题，参数、描述和 Decorator 摘要可以在需要时再显示。Semantic Zoom 根据画布缩放比例切换信息层级：近距离显示完整字段，中距离保留标题和关键参数，远距离只保留识别节点所需的信息。这两项功能都不会改变行为树资源。

节点类型还使用颜色、图标和形状进行重复编码。即使颜色不容易区分，用户仍可通过图标和轮廓识别 Root、复合节点、任务和 Decorator。插件同时提供色盲友好配色，减少只依赖单一颜色通道的问题。

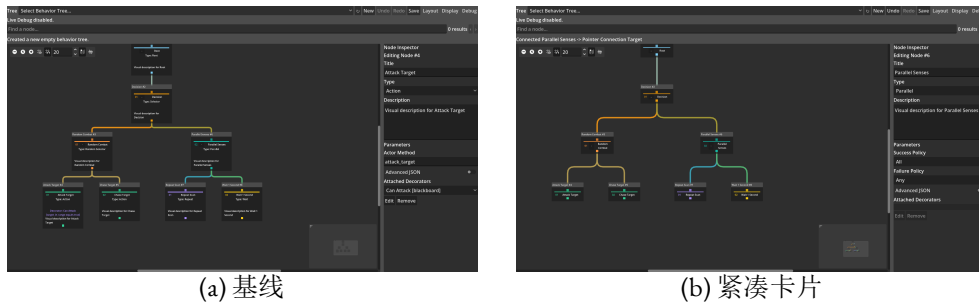


图 4.3: 同一行为树的完整细节基线和紧凑显示。

### 4.6.2 焦点与结构裁剪

鱼眼 Focus+Context 会放大指针附近的卡片，同时保留周围节点作为位置参考。最大放大倍率限制为 1.20 倍，卡片以自身中心补偿位置，避免放大后整体布局持续漂移。关闭鱼眼后，卡片恢复原来的尺寸和位置。

Search 支持标题、类型、描述、Action、Condition 和 Decorator 参数搜索，并提供上一个、下一个和清除操作。匹配节点保持明显，其他节点淡化，选定目标可以移动到当前视口。Subtree Focus 只显示指定分支和必要上下文；Context Collapse 保留折叠根和隐藏节点摘要，但暂时不绘制后代。展开后仍使用原有节点位置和父子关系。

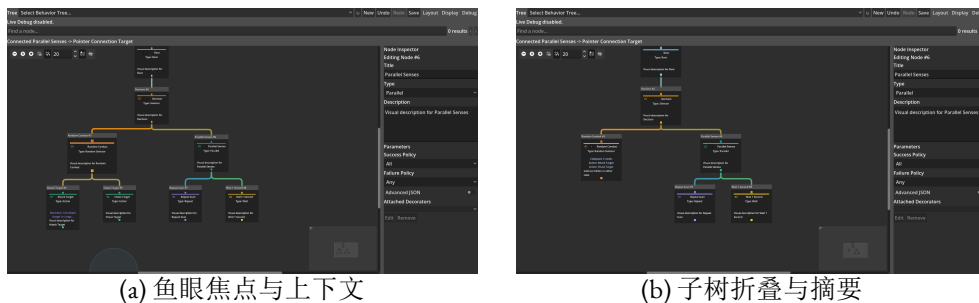


图 4.4: 局部放大和结构裁剪的两种实现。

### 4.6.3 导航与概览

增强小地图使用固定区域显示整棵树的缩略结构，并标出当前视口位置。Fit 用于把当前图形移动到可查看范围，Focus 用于定位选中节点，Show All 用于取消局部显示并返回完整树。Breadcrumb 显示从 Root 到当前节点的层级路径，路径摘要则以文字形式给出当前执行或选择路径，因此在卡片缩小时仍可确认所处位置。

搜索结果可以使用按钮或键盘在前后目标之间移动。多列布局把拥有大量子节点的分支分成多列，避免单层无限向横向扩展；稳定布局尽量保留未修改分支的位置，减少一次编辑造成的大范围移动。自动排列仍保持同级节点从左到右的执行顺序。

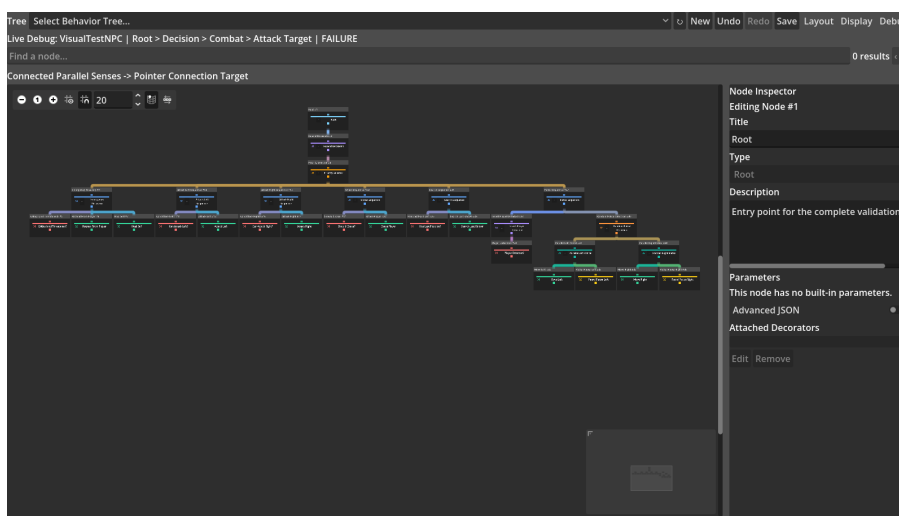


图 4.5: 复杂树中的小地图、路径摘要、导航控件和自动布局。

### 4.6.4 连线与运行表示

插件使用节点下方的输出端口和上方的输入端口表达父子关系。正式连线由插件自行绘制，拖动预览仍使用 Godot 的 GraphEdit 输入处理。正交连线使用水平和垂直线段减少曲线交叉；边捆绑让具有共同父级的连线先共享一段主干，再分向各个子节点。两种方式都保留连线点击、断开、撤销和重做。

运行时表示在同一画布上显示活动路径、当前节点、成功或失败状态、非活动分支淡化和失败原因。路径颜色沿父子关系连续传播，Decorator 失败会显示在所属节点附近。这样，用户既能看到静态结构，也能看到本次 Tick 实际经过的分支。

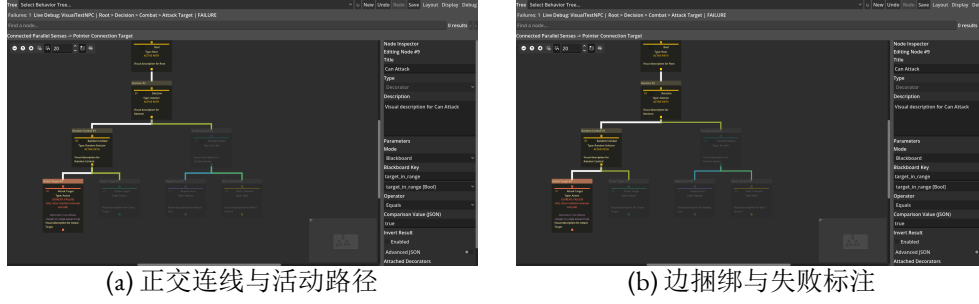


图 4.6: 连线组织和运行状态表示。

## 4.7 物理屏幕实验基础设施

屏幕测试工具读取 EDID 中的物理宽高，并将它与分辨率分开记录。对于宽  $W_{cm}$ 、高  $H_{cm}$  的屏幕，测试画布为

$$W_c = 35W_{cm}, \quad H_c = 35H_{cm},$$

其中  $W_c$  和  $H_c$  是画布的逻辑宽高。所有屏幕都使用每厘米 35 个逻辑单位，因此实验比较的是物理显示面积，而不是不同设备的像素密度。

测试窗口会移动到指定屏幕，然后加载同一棵行为树和同一种显示条件。每次测试都会保存设备信息和原始 CSV。以后接入新屏幕时，新数据写入新的 Session，不覆盖现有结果。

## 4.8 演示游戏

复杂二维竞技场用于确认行为树能够控制实际游戏敌人。玩家可以移动、跳跃、攀爬、近战、冲刺、治疗和短暂隐身。三个守卫共享同一棵 241 节点行为树。守卫会根据距离、血量、障碍物和玩家状态选择防御、近战、远程攻击、跳跃、攀爬、追击、搜索、撤退、治疗、巡逻或待机。

敌人在 330 像素内发现玩家，只有距离超过 460 像素后才会放弃目标。两个不同距离可以避免敌人在边界附近频繁切换状态。游戏使用简单图形，把开发重点放在行为逻辑和测试稳定性上。

# 5 结果与分析

## 5.1 插件功能与可玩游戏验证

插件功能测试共有 450 项检查，全部通过，如表5.1所示。日志中没有脚本错误、内存泄漏、非法访问或程序崩溃。因此，后续显示实验使用的是一个功能正确的行为树插件。

表 5.1: 最终核心自动回归。

| 套件      | 通过  | 总数  | 主要覆盖                                          |
|---------|-----|-----|-----------------------------------------------|
| 资源与运行时  | 153 | 153 | 结构、节点语义、Decorator、Actor、Schema、调试桥和资源修改后的执行复位 |
| 编辑器 GUI | 250 | 250 | 简化菜单、右键创建、图编辑、历史、Schema 与显示条件复位               |
| 基础游戏    | 13  | 13  | 三个敌人、移动、伤害、巡逻、死亡重启与生命周期                       |
| 复杂竞技场   | 34  | 34  | 索敌、响应式防御、近/远程攻击、跳跃、攀爬、搜索、撤退和恢复                |
| 合计      | 450 | 450 | 功能与游戏断言全部通过                                   |

可玩场景也通过了测试。敌人会根据情况追击、近战、防御、远程攻击、跳过障碍、攀爬梯子、搜索玩家、撤退和治疗。Actor 方法负责实际移动和伤害，241 节点行为树负责选择行为、决定执行顺序和中断低优先级行为。这部分只证明插件能够正确控制复杂敌人，不作为独立的显示优化实验。

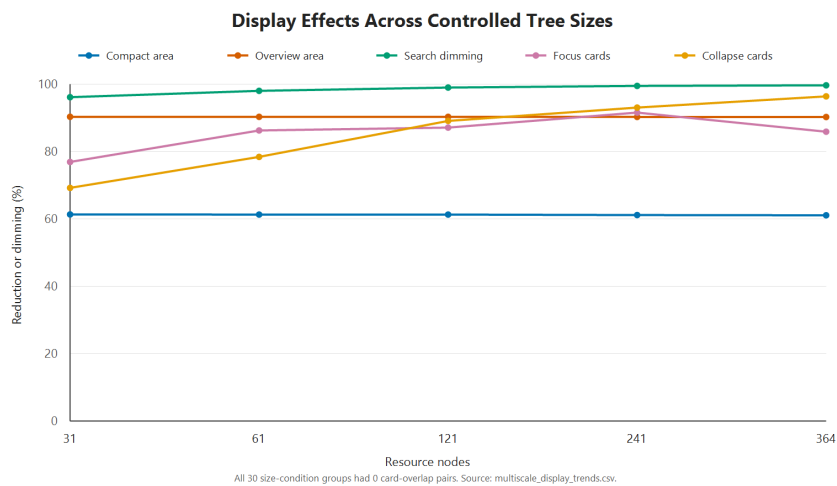
## 5.2 受控多规模显示结果

受控实验获得了计划中的 900 条数据，所有预设检查均通过。表5.2给出五种树规模的主要结果。Compact 没有隐藏节点，但把卡片总面积减少了 61.09–61.35%。Optimized Overview 同样保留全部节点，卡片总面积减少了 90.27–90.34%。这说明两种方式能够减少画面占用，但不能说明用户完成任务的速度提高了相同比例。

**表 5.2:** 五种资源规模的受控显示测量，降低比例相对 Baseline。

| 指标            | 31     | 61     | 121    | 241    | 364    |
|---------------|--------|--------|--------|--------|--------|
| 画布卡片          | 26     | 51     | 101    | 202    | 305    |
| Compact 面积降低  | 61.35% | 61.30% | 61.30% | 61.16% | 61.09% |
| Overview 面积降低 | 90.34% | 90.33% | 90.33% | 90.29% | 90.27% |
| Search 淡化     | 96.15% | 98.04% | 99.01% | 99.50% | 99.67% |
| Focus 卡片降低    | 76.92% | 86.27% | 87.13% | 91.58% | 85.90% |
| Collapse 卡片降低 | 69.23% | 78.43% | 89.11% | 93.07% | 96.39% |

Search 会突出唯一目标，并把它移入视口。非目标节点的淡化比例从 31 节点树的 96.15% 增加到 364 节点树的 99.67%。Focus 和 Collapse 会减少当前显示的上下文，但不会删除资源节点。364 节点树的 Focus 减少比例没有继续上升，是因为固定目标在这棵树中拥有较大的局部分支。所有条件都保持了节点位置和从左到右的执行顺序。

**图 5.1:** 五种受控规模下的密度、显著性和上下文降低。

### 5.3 物理屏幕尺寸结果

三块屏幕上的 270 次测试全部完成，没有失败。表 5.3 列出 241 和 364 节点树的结果。“覆盖率”表示画布能够容纳的卡片中心比例；“图/屏幕”表示完整行为树面积是屏幕测试面积的多少倍。

屏幕从 15.94 英寸增大到 31.55 英寸后，241 节点树的 Baseline 覆盖率从 18.32% 增加到 46.53%，364 节点树从 12.13% 增加到 30.82%。两种树的覆盖率都变为原来的 2.54 倍。完整树相对屏幕的面积比例则下降了 72.60%。这说明更大的物理屏幕能够显示更多上下文，但不能证明开发者一定更容易理解这些内容。



表 5.3: 三种物理屏幕尺寸下的复杂树覆盖率。分辨率仅为审计元数据，不是实验处理。

| 屏幕                | 表面 (cm) | 画布        | 241 覆盖率 | 364 覆盖率 | 241 图/屏幕 | 364 图/屏幕 |
|-------------------|---------|-----------|---------|---------|----------|----------|
| BOE0CD1, 15.94 英寸 | 34×22   | 1190×770  | 18.32%  | 12.13%  | 116.38x  | 184.13x  |
| H27T22S, 26.96 英寸 | 60×33   | 2100×1155 | 38.61%  | 25.57%  | 43.96x   | 69.56x   |
| U32G3X, 31.55 英寸  | 70×39   | 2450×1365 | 46.53%  | 30.82%  | 31.89x   | 50.45x   |

显示优化在三块屏幕上的相对效果基本一致。Compact 的面积减少比例保持在 61.09–61.35%，Overview 保持在 90.27–90.34%。Search 在全部 15 个“屏幕–规模”组合中都把目标放入视口。所有条件也都保持了节点位置和执行顺序。

结构裁剪对小屏幕的影响更明显。对于 241 节点树，三块屏幕上的 Subtree Focus 覆盖率依次为 82.35%、88.24% 和 88.24%，Context Collapse 为 50.00%、64.29% 和 64.29%。对于 364 节点树，Subtree Focus 覆盖率依次为 86.05%、95.35% 和 95.35%，Context Collapse 为 36.36%、54.55% 和 54.55%。这里的覆盖率表示当前条件保留的卡片中有多少进入画布，不表示完整行为树有相同比例同时可见。

5.4 对应研究问题的结果

功能测试和可玩游戏确认了实验平台能够正确编写、保存和执行行为树。这是显示实验成立的前提，不是本文要单独研究的结果。

五种节点规模和三种屏幕尺寸的数据表明，六种显示条件能够稳定改变卡片密度、显示上下文和搜索目标的显著程度，同时保持节点位置、父子关系和执行顺序。实验数据支持这些优化改善了大型行为树的部分可测显示和编辑交互表现。由于没有真人参与者，本研究不能进一步声称人的可读性或使用效率已经得到证明。

## 6.1 实验平台的功能正确性

完整插件是显示优化实验的平台，不是单独的研究对象。它能够保存和运行行为树，也支持 Actor、复合节点、条件、动作、Decorator、黑板和实时调试。因此，实验使用的是真正能够运行的行为树，而不是只画在画布上的静态节点图。

资源与运行时、编辑器界面、基础游戏和复杂竞技场共 450 项检查全部通过。241 节点行为树能够控制敌人的索敌、防御、近战、远程攻击、跳跃、攀爬、搜索、撤退和治疗。这些结果说明插件能够为显示优化实验提供功能正确、具有实际游戏行为的测试平台。

## 6.2 显示优化效果评估

### 6.2.1 不同节点规模下的优化效果

Compact 在五种规模下保留全部卡片，同时把卡片总面积减少 61.09–61.35%。各规模的降低比例非常接近，说明紧凑卡片的效果不会随着节点数量增加而明显减弱。它适合在仍需查看全部节点时减少单张卡片占用的空间。

Optimized Overview 同样保留全部卡片，面积降低稳定在 90.27–90.34%。与 Compact 相比，它进一步减少卡片内部信息，因此更适合查看大型行为树的整体分支结构。31 到 364 节点之间的结果变化很小，说明该组合在不同树规模下都能保持相近的信息密度改善。

Optimized Search 的非目标淡化比例从 31 节点树的 96.15% 增加到 364 节点树的 99.67%。随着行为树变大，画面中的非目标节点更多，搜索对目标显著性的提升也更明显。实验中的固定目标在所有规模下都能进入视口，说明 Search 能够在大型树中稳定完成已知节点定位。

Subtree Focus 在五种规模下减少了 76.92–91.58% 的显示卡片，364 节点树为 85.90%。该比例由目标分支本身的大小决定，所以不会随总节点数严格递增。Context Collapse 的卡片减少比例从 69.23% 增加到 96.39%，说明树越大，可折叠的非目标分支越多。两种方法都能减少当前画面中的无关上下文，但 Focus 面向指定分支，Collapse 则保留更多整体结构位置。

综合五种规模的数据，Compact 和 Overview 的面积降低保持稳定，Search 的目标突出效果随规模增加，Focus 和 Collapse 则直接减少同时显示的上下文。这些结果说明显示优化在小型树中已经有效，在 121、241 和 364 节点的大型树中作用更加明显。

### 6.2.2 不同物理屏幕尺寸下的优化效果

屏幕尺寸实验显示，物理显示面积会直接影响大型树能够呈现的上下文。屏幕从 15.94 英寸增大到 31.55 英寸后，241 节点树的 Baseline 覆盖率从 18.32% 增加到 46.53%，364 节点树从 12.13% 增加到 30.82%。两种复杂树的覆盖率都变为小屏幕的 2.54 倍，说明更大的物理屏幕能够同时容纳更多节点。

不同功能对屏幕尺寸的反应并不相同。Compact 在三块屏幕上的卡片面积降低均为 61.09–61.35%，Overview 均为 90.27–90.34%。两者在小屏和大屏上的相对效果几乎相同，因此提供的是不依赖屏幕尺寸的信息密度改进，没有额外放大大屏幕优势。由于 241 和 364 节点树已经达到 0.10 倍的最小缩放，它们虽然明显缩小卡片和信息量，却没有让小屏幕一次显示更多完整树节点。

Subtree Focus 对小屏幕的补偿最明显。241 节点树在小屏幕上增加 64.04 个百分点，在大屏幕上增加 41.70 个百分点；小屏与大屏的差距由 28.22 缩小到 5.88 个百分点。364 节点树的小屏增加 73.92 个百分点，大屏增加 64.53 个百分点，屏幕间差距由 18.69 缩小到 9.30 个百分点。Context Collapse 的补偿较弱：241 节点树的屏幕间差距缩小到 14.29 个百分点，364 节点树只缩小到 18.18 个百分点。原因是 Collapse 仍保留路径和折叠根，而 Focus 只保留当前分支及必要上下文。这里比较的是当前条件保留卡片的覆盖率，不表示完整树已经全部显示。

Search 在全部 15 个“屏幕–规模”组合中都把固定目标移入视口，说明屏幕变小没有破坏目标定位。综合来看，大屏幕仍能显示更多绝对上下文，但优化没有进一步扩大这种优势。Compact 和 Overview 跨屏幕保持稳定，Search 保持目标可定位，Focus 则明显缩小小屏与大屏的覆盖差距。因此，数据更支持“显示优化补偿了小屏幕的部分空间限制，同时保留大屏幕的绝对容量优势”，而不是“显示优化主要扩大大屏幕优势”。这一结论只针对卡片面积、上下文覆盖和目标定位，不等同于人的可读性或使用效率。

# 7 结论

## 7.1 总结

本文在 Godot 4.6 中实现了一个可视化行为树插件，并研究大型行为树的显示问题。Godot 提供了通用图编辑控件，但没有完整的行为树编辑和运行流程。传统节点图在节点、参数和调试信息增加后，也会占用很大的画布空间。

最终插件能够创建、保存、加载和运行行为树，支持黑板、Decorator、多种复合节点、Action、Condition、撤销、重做和 Live Debug。复杂二维场景使用 241 节点行为树控制敌人的索敌、防御、近战、远程攻击、跳跃、攀爬、搜索、撤退、治疗和巡逻。

功能测试和可玩游戏确认插件能够作为实验平台。本文真正研究的是显示优化，而不是行为树基础功能本身。

五种树规模的实验表明, Compact 把卡片面积减少了 61.09–61.35%, Overview 减少了 90.27–90.34%, Search 淡化了 96.15–99.67% 的非目标卡片。Focus 和 Collapse 也明显减少了当前显示的上下文，所有条件都没有改变保存位置和执行顺序。

三块屏幕的实验表明，31.55 英寸屏幕上复杂树的覆盖率是 15.94 英寸屏幕的 2.54 倍。Compact、Overview 和 Search 在不同屏幕上的作用保持稳定。总体来看，显示优化改善了大型行为树的部分客观显示和编辑交互指标。由于没有真人实验，本文不能证明用户理解能力已经提高。

## 7.2 贡献

本项目的主要贡献包括：一个能够实际运行的 Godot 行为树插件；六种定义清楚且可以恢复到 Baseline 的显示条件；以及包含 900 条多规模数据、270 条物理屏幕数据、自动功能测试、原始文件校验值和固定测试资源的实验材料。

实验数据表明，各项显示功能解决的问题不同：Compact 和 Overview 改善信息密度，Search 改善目标定位，Focus 和 Collapse 减少当前上下文，物理大屏幕则提供更多显示面积。

### 7.3 局限

实验只在一台 Windows 电脑、Godot 4.6、一个渲染环境和三块现有屏幕上进行。最大生成树包含 364 个节点。自动实验能够测量界面变化，但不能证明长期开发效率或认知负担。当前 Live Debug 只显示最新状态，没有完整执行时间线。Service、可复用子树、Asset Library 发布和跨版本测试也不在本项目的完成范围内。

### 7.4 未来工作

以后可以接入更多物理屏幕，并在其他 Godot 版本、GPU 和不规则人工行为树上重复实验。如果具备伦理审批、参与者和时间，也可以设计真人任务，测量节点定位、分支理解和编辑操作。真人实验属于后续研究，不是当前论文已经获得的数据。

插件以后可以增加可复用子树、暂停和单步调试、执行时间线，以及多个 Actor 的调试选择。显示研究还可以加入平滑的布局变化，并测试用户在切换显示方式后能否重新找到原来的节点。正式发布前还需要完成跨版本验证和 Asset Library 准备。

### 7.5 最终结论

本项目完成了一个可以实际用于游戏开发的 Godot 行为树插件，并为大型行为树加入了可独立开关的显示优化。自动实验表明，这些功能能够减少卡片面积、突出搜索目标和控制当前显示范围，同时不改变行为树结构和执行顺序。它既可以继续作为游戏 AI 编辑工具，也可以作为以后进行真人可用性研究的平台。

## 参考文献

- Colledanchise, M. and Ögren, P. (2018). *Behavior Trees in Robotics and AI: An Introduction*. CRC Press. doi:10.1201/9780429489105.
- Furnas, G. W. (1986). Generalized fisheye views. *Proceedings of CHI '86*, 16–23. doi:10.1145/22339.22342.
- Lamping, J., Rao, R. and Pirolli, P. (1995). A focus+context technique based on hyperbolic geometry for visualizing large hierarchies. *Proceedings of CHI '95*, 401–408. doi:10.1145/223904.223956.
- Cockburn, A., Karlson, A. and Bederson, B. B. (2008). A review of overview+detail, zooming, and focus+context interfaces. *ACM Computing Surveys*, 41(1), 1–31. doi:10.1145/1456650.1456652.
- Song, H., Kim, B., Lee, B. and Seo, J. (2010). A comparative evaluation on tree visualization methods for hierarchical structures with large fan-outs. *Proceedings of CHI 2010*, 223–232. doi:10.1145/1753326.1753359.
- Bae, J. and Watson, B. (2011). Developing and evaluating Quilts for the depiction of large layered graphs. *IEEE TVCG*, 17(12), 2268–2277. doi:10.1109/TVCG.2011.187.
- Dogrusoz, U. et al. (2018). Efficient methods and readily customizable libraries for managing complexity of large networks. *PLOS ONE*, 13(5), e0197238. doi:10.1371/journal.pone.0197238.
- Misue, K. et al. (1995). Layout adjustment and the mental map. *Journal of Visual Languages and Computing*, 6(2), 183–210. doi:10.1006/jvlc.1995.1010.
- Reingold, E. M. and Tilford, J. S. (1981). Tidier drawings of trees. *IEEE Transactions on Software Engineering*, SE-7(2), 223–228.
- Plaisant, C. et al. (2002). SpaceTree. *Proceedings of IEEE InfoVis 2002*, 57–64.
- Shneiderman, B. (1996). The eyes have it. *Proceedings of IEEE Visual Languages 1996*, 336–343.

Godot Engine contributors (2026a). GraphEdit class reference. [https://docs.godotengine.org/en/stable/classes/class\\_graphedit.html](https://docs.godotengine.org/en/stable/classes/class_graphedit.html).

Godot Engine contributors (2026b). EditorPlugin class reference. [https://docs.godotengine.org/en/stable/classes/class\\_editorplugin.html](https://docs.godotengine.org/en/stable/classes/class_editorplugin.html).

Epic Games (2026). Behavior Tree in Unreal Engine: Overview. <https://dev.epicgames.com/documentation/en-us/unreal-engine/behavior-tree-in-unreal-engine---overview>.

# 六种实验条件参考

表 A.1: 论文使用的六种显示实验条件。

| 条件                 | 开关与输入                              | 实际显示效果               |
|--------------------|------------------------------------|----------------------|
| Baseline           | 关闭实验优化，展开全部节点，清空搜索和聚焦，缩放 100%。     | 完整卡片和全部分支。           |
| Compact Cards      | 只开启 Compact。                       | 全部节点保留，卡片只显示类型和短标题。  |
| Optimized Overview | 开启 Compact 与 Semantic Zoom，缩放 50%。 | 全部节点保留，使用最低细节查看整体结构。 |
| Optimized Search   | 在 Overview 上搜索固定唯一标题。              | 目标进入视口并保持显著，其他卡片淡化。  |
| Subtree Focus      | 开启 Compact，设置固定 Focus Root。        | 只显示指定分支及必要上下文。       |
| Context Collapse   | 开启 Compact，折叠固定目标路径之外的指定分支。        | 保留折叠根与摘要，隐藏其后代卡片。    |

六种条件的详细步骤和测量目的见表3.1。插件中的其他显示功能不属于本论文实验条件。



# 复现与证据索引

以下路径均相对于仓库根目录：

活动项目： `testgame/testgame/`

分发插件： `visual_scripting/addons/behavior_tree_editor/`

复杂树： `testgame/testgame/behavior_trees/complex_guard_validation_tree.tres`

测试脚本： `testgame/testgame/tests/`

多规模显示数据： `testgame/testgame/test_results/multiscale_display_raw.csv`

多规模显示汇总： `testgame/testgame/test_results/multiscale_display_summary.csv`

多规模显示脚本： `testgame/testgame/tests/run_multiscale_display_experiment.gd`

物理屏幕证据： 实验根目录为 `research/display_optimization/physical_screen_size_experiment/`；其中 `data/2026-08-23_three_displays/Session` 包含 `combined_raw.csv`、`screen_size_comparison.csv` 与 `report_zh.md`。

发布包： `dist/behavior-tree-editor-0.9.0.zip`

# 初稿完成清单

正式提交前，作者应：替换身份与导师占位符；按学校政策修改声明并如实说明 AI 协助；在 Tabula 核实截止时间和词数规则；请导师审阅文献、研究问题和证据边界；按院系要求执行 Turnitin；用模板编译并逐页检查；加入最终词数和要求的仓库/归档信息。